Lab Assignment No.	08
Title	Implement DEAP (Distributed Evolutionary Algorithms) using Python.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 08

Title: Implementation of DEAP (Distributed Evolutionary Algorithms in Python) Using Python

Problem Statement: To design and implement Distributed Evolutionary Algorithms (DEAP) using Python in Jupyter Notebook for solving complex optimization problems through evolutionary computation techniques.

Prerequisites:

- Basic knowledge of Python programming
- Understanding of evolutionary algorithms and optimization techniques
- Familiarity with genetic algorithms and related bio-inspired methods
- Basic knowledge of Jupyter Notebook environment

Software Requirements:

- Jupyter Notebook
- Python 3.x
- Required Python libraries: DEAP, NumPy, Matplotlib (for visualization)

Hardware Requirements:

- A computer with minimum 4 GB RAM
- Dual-core processor or higher
- Stable internet connection (optional for additional resources)

Theory: The Distributed Evolutionary Algorithms in Python (DEAP) framework is an advanced library designed to facilitate the rapid prototyping and implementation of evolutionary algorithms. DEAP supports a wide range of bio-inspired algorithms, including genetic algorithms, genetic programming, evolution strategies, and more. It is highly flexible, allowing users to customize the algorithm components to address specific problem domains.

Key Concepts:

1. **Evolutionary Algorithms (EAs):** A family of optimization algorithms inspired by the process of natural selection, involving operations like selection, crossover, and mutation.

2. **Genetic Algorithms (GAs):** A type of EA where candidate solutions (individuals) evolve over generations to find optimal or near-optimal solutions.
3. **Selection Mechanism:** Identifies the fittest individuals based on a fitness function to participate in reproduction.
4. **Crossover (Recombination):** Combines genetic information from two parent individuals to create offspring with mixed characteristics.
5. **Mutation:** Introduces random changes to individual solutions, promoting genetic diversity and preventing premature convergence.
6. **Fitness Evaluation:** Measures the quality of each candidate solution based on a problem-specific objective function.

Mathematical Model:

- **Fitness Function:**
- **Selection Probability:** Based on fitness proportionate selection or tournament selection methods.
- **Crossover Rate (Pc):** Defines the likelihood of performing crossover between parent individuals.
- **Mutation Rate (Pm):** Determines the probability of applying random mutations to offspring.

Applications:

- Optimization problems in engineering and science
- Machine learning model tuning
- Scheduling and resource allocation
- Robotics path planning
- Game strategy optimization

Source Code –

```
# Import Required Libraries

import random from deap

import base, creator, tools, algorithms
```

```
import numpy as np

# Define the evaluation function (minimize a simple mathematical function)

def eval_func(individual):

# Example evaluation function (minimize a quadratic function)

return sum(x ** 2 for x in individual),

 # DEAP setup

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))

creator.create("Individual", list, fitness=creator.FitnessMin)

 toolbox = base.Toolbox()

# Define attributes and individuals

 toolbox.register("attr_float", random.uniform, -5.0, 5.0) # Example: Float values between -5 and
5

toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_float, n=3) #
Example: 3-dimension

toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# Evaluation function and genetic operators

toolbox.register("evaluate", eval_func)

toolbox.register("mate", tools.cxBlend, alpha=0.5)

toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

# Create population

 population = toolbox.population(n=50)

 # Genetic Algorithm

parameters generations = 20

 # Run the algorithm

for gen in range(generations):

offspring = algorithms.varAnd(population, toolbox, cxpb=0.5, mutpb=0.1)
```

```
fits = toolbox.map(toolbox.evaluate, offspring)

for fit, ind in zip(fits, offspring):

ind.fitness.values = fit

population = toolbox.select(offspring, k=len(population))

# Get the best individual after generations

best_ind = tools.selBest(population, k=1)[0]

 best_fitness = best_ind.fitness.values[0]

print("Best individual:", best_ind)

print("Best fitness:", best_fitness)
```

Conclusion: Through this practical, we have explored the implementation of Distributed Evolutionary Algorithms using the DEAP library in Python. By leveraging evolutionary principles such as selection, crossover, and mutation, we demonstrated the effectiveness of DEAP in solving complex optimization problems. This practical highlighted the flexibility and power of DEAP in developing custom evolutionary algorithms for diverse applications.